Lesson 2 Notes: Coordinate List Representation and Creation

1. Main idea of the lesson

Lesson 2 explains how to store a sparse matrix using **Coordinate List representation**.

A sparse matrix is a matrix that has mostly zero values.

Instead of storing every value in the matrix, we store only the values that are not zero.

But there is one important difference from a diagonal matrix:

- In a diagonal matrix, useful values appear only where `i == j`.
- In a sparse matrix, useful values can appear anywhere.

So for every non-zero value, we must store three things:

```
row number
column number
value
```

This method is called **Coordinate List representation**.

It is also commonly called **COO representation**.

---

2. Quick connection to Lesson 1

In Lesson 1, you learned that we do not always need to store a full matrix.

For a diagonal matrix, only the diagonal values are important.

Example:

```
3 0 0
0 7 0
0 0 5
```

Instead of storing all 9 values, we store only:

```
A = [3, 7, 5]
```

That works because diagonal values follow a fixed pattern:

```
i == j
```

But sparse matrices are different.

Example:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

Here, the non-zero values do not follow a simple diagonal pattern.

So we cannot just store the values alone.

We must also store their positions.

---

3. What is a sparse matrix?

A sparse matrix is a matrix where most values are zero.

Example:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

This matrix has:

```
4 rows
5 columns
```

So it is a:

```
4 × 5 matrix
```

A normal 2D array would store all positions:

```
4 × 5 = 20 values
```

But only 5 values are non-zero:

```
7, 8, 5, 9, 6
```

So storing all 20 values wastes memory.

Sparse storage avoids storing unnecessary zeros.

---

4. Why sparse matrix storage is useful

In a normal matrix, every value is stored, even if it is zero.

For example:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

Normal storage stores:

```
20 total values
```

Sparse storage stores only:

```
5 non-zero values
```

But each stored value needs its position.

So we store:

```
row, column, value
```

This is still usually much smaller than storing the full matrix when the matrix contains mostly zeros.

---

5. Why we need row and column numbers

In a diagonal matrix, if we store this:

```
A = [3, 7, 5]
```

we know the values belong to:

```
M[1,1]
M[2,2]
M[3,3]
```

because the rule is:

```
i == j
```

But in a sparse matrix, values can appear anywhere.

Example:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

If we store only:

```
[7, 8, 5, 9, 6]
```

that is not enough.

The computer would not know where those values belong.

So we store:

```
(1, 3, 7)
(2, 5, 8)
(3, 2, 5)
(4, 1, 9)
(4, 5, 6)
```

Each record means:

```
(row, column, value)
```

---

6. Coordinate List representation

Coordinate List representation stores each non-zero element using its coordinates.

The word **coordinate** means position.

In a matrix, a position is described using:

```
row number
column number
```

So each non-zero element is stored as:

```
row, column, value
```

For example:

```
(1, 3, 7)
```

means:

```
At row 1, column 3, the value is 7.
```

---

7. Coordinate List table example

Consider this sparse matrix:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

Using 1-based matrix positions, the Coordinate List form is:

Row	Column	Value
1	3	7
2	5	8
3	2	5
4	1	9
4	5	6

This table stores only the non-zero values.

All positions not shown in the table are treated as zero.

So this table is enough to represent the whole matrix.

---

8. Meaning of one Coordinate List row

A row in the Coordinate List table looks like this:

```
1 3 7
```

This means:

```
row = 1
column = 3
value = 7
```

So the matrix has:

```
M[1,3] = 7
```

Another example:

```
4 5 6
```

This means:

```
row = 4
column = 5
value = 6
```

So:

```
M[4,5] = 6
```

---

9. Important indexing note

In this lesson, matrix positions are written using **1-based indexing**.

That means rows and columns start from 1.

Example:

```
M[1,1]
M[1,2]
M[2,1]
```

But arrays in C and C++ use **0-based indexing**.

That means array indexes start from 0.

Example:

```
e[0]
e[1]
e[2]
```

So the first non-zero element entered by the user is stored in:

```
e[0]
```

The second non-zero element is stored in:

```
e[1]
```

The third non-zero element is stored in:

```
e[2]
```

This means the matrix position and the array index are not the same thing.

---

10. The `Element` structure

Each non-zero value needs three pieces of information:

```
row number
column number
value
```

So we create a structure called `Element`.

```
struct Element {
    int i;  // row number
    int j;  // column number
    int x;  // value
};
```

Meaning:

Field	Meaning
`i`	row number
`j`	column number
`x`	actual value stored at that row and column

One `Element` represents one non-zero value.

It does not represent the whole matrix.

---

11. Example of one `Element`

Suppose we write:

```
Element e;
e.i = 1;
e.j = 3;
e.x = 7;
```

This means:

```
Matrix[1][3] = 7
```

So this one `Element` stores this information:

```
row = 1
column = 3
value = 7
```

A sparse matrix will usually need many `Element` records.

Each one stores one non-zero value.

---

12. Why `Element` is needed

Without the `Element` structure, we would need separate arrays like this:

```
row[]
column[]
value[]
```

But using a structure is cleaner.

Instead of separating related data, we group them together.

One non-zero value becomes one record:

```
Element = row + column + value
```

So the code becomes easier to understand.

---

13. The `Sparse` structure

The `Element` structure stores one non-zero value.

But we also need a structure for the whole sparse matrix.

That structure is called `Sparse`.

```
struct Sparse {
    int m;              // total number of rows
    int n;              // total number of columns
    int num;            // number of non-zero elements
    struct Element *e;  // dynamic array of Element records
};
```

This structure stores the important information about the full matrix.

---

14. Meaning of the fields in `Sparse`

Field	Meaning
`m`	total number of rows in the matrix
`n`	total number of columns in the matrix
`num`	number of non-zero elements
`e`	pointer to a dynamic array of `Element` records

Example:

For this matrix:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

we have:

```
m = 4
n = 5
num = 5
```

because:

```
the matrix has 4 rows
the matrix has 5 columns
the matrix has 5 non-zero values
```

---

15. `num` does not mean total matrix size

A common beginner mistake is thinking:

```
num = m × n
```

That is wrong.

`m × n` gives the total number of positions in the full matrix.

But `num` means only the number of non-zero elements.

For example:

```
m = 4
n = 5
```

So the full matrix has:

```
m × n = 4 × 5 = 20 positions
```

But if only 5 values are non-zero:

```
num = 5
```

So:

```
m × n = total visible positions
num = actual stored non-zero values
```

---

16. What the pointer `e` stores

Inside the `Sparse` structure, we have:

```
struct Element *e;
```

This means `e` is a pointer to an array of `Element` records.

If `num = 5`, then `e` will point to an array like this:

```
e[0]
e[1]
e[2]
e[3]
e[4]
```

Each item stores one non-zero element.

Example:

```
e[0] = {1, 3, 7}
e[1] = {2, 5, 8}
e[2] = {3, 2, 5}
e[3] = {4, 1, 9}
e[4] = {4, 5, 6}
```

So `e` is the actual storage area for the non-zero elements.

---

17. Why dynamic allocation is needed

Before the user enters the matrix, the program does not know how many non-zero values there will be.

The user might enter:

```
num = 3
```

or:

```
num = 10
```

or:

```
num = 100
```

So the program should not create a fixed-size array blindly.

Instead, it first reads `num`.

Then it creates exactly enough space.

In C++ style:

```
s->e = new Element[s->num];
```

If `s->num = 5`, this creates space for 5 `Element` records.

---

18. Why we do not allocate `m * n` elements

A sparse matrix is useful because we avoid storing zeros.

So this is not a good idea:

```
s->e = new Element[s->m * s->n];
```

That would allocate space based on the full matrix size.

But the whole point is to store only non-zero values.

Correct allocation:

```
s->e = new Element[s->num];
```

This means:

```
Create space only for the non-zero elements.
```

---

19. Stack object and heap array

In `main`, we usually create the sparse matrix object like this:

```
struct Sparse s;
```

This creates `s` as a normal local variable.

Its fields are:

```
s.m
s.n
s.num
s.e
```

But the array of elements is created dynamically:

```
s.e = new Element[s.num];
```

So the structure itself is a local object, but the element array is created in dynamic memory.

Beginner mental model:

```
Sparse s stores general information.
s.e points to the actual list of non-zero elements.
```

---

20. Why we pass by address

The create function must change the original sparse matrix.

It must fill:

```
m
n
num
e
```

So we pass the address of the sparse matrix to the function.

Function form:

```
void create(struct Sparse *s)
```

Function call:

```
create(&s);
```

Here:

```
&s means address of s
```

So `create()` receives access to the original matrix, not just a copy.

---

21. Why we use `->` inside `create()`

Inside the function:

```
void create(struct Sparse *s)
```

`s` is a pointer.

So we access fields using `->`.

Correct:

```
s->m
s->n
s->num
s->e
```

Wrong:

```
s.m
s.n
s.num
s.e
```

The dot operator `.` is used when we have the actual object.

The arrow operator `->` is used when we have a pointer to the object.

Simple rule:

```
Object: use .
Pointer: use ->
```

---

22. Basic create function

Here is a beginner-friendly C++ style version of the `create()` function.

```cpp
#include <iostream>
using namespace std;

struct Element {
    int i;
    int j;
    int x;
};

struct Sparse {
    int m;
    int n;
    int num;
    struct Element *e;
};

void create(struct Sparse *s) {
    cout << "Enter matrix dimensions: ";
    cin >> s->m >> s->n;

    cout << "Enter number of non-zero elements: ";
    cin >> s->num;
```

```
    s->e = new Element[s->num];

    cout << "Enter row, column, and value for each non-zero element:\n";

    for (int k = 0; k < s->num; k++) {
        cin >> s->e[k].i >> s->e[k].j >> s->e[k].x;
    }
}
```

---

23. What the create function does

The `create()` function does these steps:

1. Reads the number of rows.
2. Reads the number of columns.
3. Reads the number of non-zero elements.
4. Allocates a dynamic array of `Element` records.
5. Reads every non-zero element.
6. Stores each non-zero element as row, column, and value.

So this function builds the compact sparse matrix representation.

---

24. Step-by-step explanation of `create()`

Step 1: Read the matrix dimensions

```
cin >> s->m >> s->n;
```

This reads:

```
number of rows
number of columns
```

Example:

```
4 5
```

Then:

```
s->m = 4
s->n = 5
```

Step 2: Read the number of non-zero elements

```
cin >> s->num;
```

Example:

```
5
```

Then:

```
s->num = 5
```

Step 3: Allocate memory

```
s->e = new Element[s->num];
```

If:

```
s->num = 5
```

then the program creates:

```
s->e[0]
s->e[1]
s->e[2]
s->e[3]
s->e[4]
```

Step 4: Read each non-zero element

```
for (int k = 0; k < s->num; k++) {
    cin >> s->e[k].i >> s->e[k].j >> s->e[k].x;
}
```

The loop runs once for each non-zero value.

If `num = 5`, the loop runs 5 times.

The variable `k` is the array index.

It starts at 0 because arrays in C and C++ start at 0.

---

25. Full dry run

Suppose the user enters:

```
Enter matrix dimensions: 4 5
Enter number of non-zero elements: 5

1 3 7
2 5 8
3 2 5
4 1 9
4 5 6
```

At the start, in `main`, we have:

```
struct Sparse s;
```

Before calling `create()`, the fields may contain garbage values because they are not initialized yet.

Then we call:

```
create(&s);
```

Now `create()` can modify the original `s`.

---

26. Dry run: reading dimensions

Input:

```
4 5
```

Code:

```
cin >> s->m >> s->n;
```

Result:

```
s->m = 4
s->n = 5
```

So the matrix has:

```
4 rows
5 columns
```

---

27. Dry run: reading `num`

Input:

```
5
```

Code:

```
cin >> s->num;
```

Result:

```
s->num = 5
```

This means:

```
There are 5 non-zero elements.
```

---

28. Dry run: allocating memory

Code:

```
s->e = new Element[s->num];
```

Since:

```
s->num = 5
```

this becomes:

```
s->e = new Element[5];
```

Now the program has space for 5 non-zero elements:

```
s->e[0]
s->e[1]
s->e[2]
s->e[3]
s->e[4]
```

---

29. Dry run: loop iteration `k = 0`

Input:

```
1 3 7
```

Code:

```
cin >> s->e[0].i >> s->e[0].j >> s->e[0].x;
```

Stored result:

```
s->e[0].i = 1
s->e[0].j = 3
s->e[0].x = 7
```

Meaning:

```
M[1,3] = 7
```

---

30. Dry run: loop iteration `k = 1`

Input:

```
2 5 8
```

Stored result:

```
s->e[1].i = 2
s->e[1].j = 5
s->e[1].x = 8
```

Meaning:

```
M[2,5] = 8
```

---

31. Dry run: loop iteration `k = 2`

Input:

```
3 2 5
```

Stored result:

```
s->e[2].i = 3
s->e[2].j = 2
s->e[2].x = 5
```

Meaning:

```
M[3,2] = 5
```

---

32. Dry run: loop iteration `k = 3`

Input:

```
4 1 9
```

Stored result:

```
s->e[3].i = 4
s->e[3].j = 1
s->e[3].x = 9
```

Meaning:

```
M[4,1] = 9
```

---

33. Dry run: loop iteration `k = 4`

Input:

```
4 5 6
```

Stored result:

```
s->e[4].i = 4
s->e[4].j = 5
s->e[4].x = 6
```

Meaning:

```
M[4,5] = 6
```

---

34. Final stored representation

After all input is complete, the matrix is stored as:

```
s->m = 4
s->n = 5
s->num = 5
```

The element array stores:

```
s->e[0] = {1, 3, 7}
s->e[1] = {2, 5, 8}
s->e[2] = {3, 2, 5}
s->e[3] = {4, 1, 9}
s->e[4] = {4, 5, 6}
```

This compact representation stands for this full matrix:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

The zeros are not stored.

They are understood logically.

---

35. Visual comparison: full matrix vs stored form

Full matrix view:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

Actual stored form:

```
Rows:    4
Columns: 5
Non-zero count: 5

Index  Row  Column  Value
0      1    3       7
1      2    5       8
2      3    2       5
3      4    1       9
4      4    5       6
```

The full matrix has 20 positions.

The sparse representation stores only 5 records.

---

36. Important assumption: row-major order

For now, it is best to enter non-zero elements in row-major order.

Row-major order means:

```
Sort by row first.
If two elements are in the same row, sort by column.
```

Good order:

```
1 3 7
2 5 8
3 2 5
4 1 9
4 5 6
```

This order goes from top to bottom in the matrix.

If two values are in the same row, it goes from left to right.

This becomes very important in later lessons when we display and add sparse matrices efficiently.

---

37. Beginner mental model

Think of the full matrix as a big grid.

Most places in the grid are empty.

Instead of storing every empty place, we store only the places that contain something.

For example:

```
(1, 3, 7)
```

means:

```
Go to row 1, column 3. Put 7 there.
```

If a position is not listed, it is zero.

So the Coordinate List is like a list of important locations.

---

38. Another mental model: map locations

Imagine a map with many empty fields.

You do not write down every empty field.

You only write down important places:

```
House at location A
Shop at location B
School at location C
```

Coordinate List works the same way.

For a sparse matrix, we write:

```
Value 7 at row 1, column 3
Value 8 at row 2, column 5
Value 5 at row 3, column 2
```

Everything else is treated as empty, or zero.

---

39. Complexity of creation

The `create()` function reads each non-zero element once.

If there are `num` non-zero elements, the loop runs `num` times.

So the time complexity is:

```
O(num)
```

This means the time depends on the number of non-zero elements, not the full matrix size.

---

40. Space complexity of Coordinate List storage

The dynamic array stores exactly `num` `Element` records.

So the space complexity is:

```
O(num)
```

A normal matrix would require:

```
O(m × n)
```

Coordinate List storage requires:

```
O(num)
```

This is useful when:

```
num is much smaller than m × n
```

---

41. Example of memory saving

Suppose we have a matrix with:

```
1000 rows
1000 columns
```

A normal matrix has:

```
1000 × 1000 = 1,000,000 positions
```

If only 200 values are non-zero, then:

```
num = 200
```

Normal storage stores 1,000,000 values.

Coordinate List storage stores only 200 records.

Each record stores:

```
row
column
value
```

Even though each record has three fields, this is still much smaller than storing one million positions.

---

42. Complexity table

Operation	Time Complexity	Space Complexity	Reason
Create sparse matrix	`O(num)`	`O(num)`	Reads and stores each non-zero element once
Store full normal matrix	-	`O(m × n)`	Stores every matrix position

Operation	Time Complexity	Space Complexity	Reason
Store Coordinate List matrix	-	`O(num)`	Stores only non-zero elements

---

43. Common beginner mistake 1: confusing `num` with `m × n`

Wrong idea:

```
num means total number of positions
```

Correct idea:

```
num means total number of non-zero elements
```

Example:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

Here:

```
m = 4
n = 5
m × n = 20
num = 5
```

So `num` is not 20.

It is 5.

---

44. Common beginner mistake 2: allocating too much memory

Wrong:

```
s->e = new Element[s->m * s->n];
```

This creates space for every position.

That defeats the purpose of sparse storage.

Correct:

```
s->e = new Element[s->num];
```

This creates space only for non-zero elements.

---

45. Common beginner mistake 3: using `.` instead of `->`

Inside this function:

```
void create(struct Sparse *s)
```

`s` is a pointer.

So this is correct:

```
s->m
s->n
s->num
s->e
```

This is wrong inside `create()`:

```
s.m
s.n
s.num
s.e
```

Use this rule:

```
If you have an object, use dot.
If you have a pointer, use arrow.
```

---

46. Common beginner mistake 4: storing zeros

Wrong idea:

```
Store every value, including zeros.
```

Correct idea:

```
Store only non-zero values.
```

If a value is zero, we do not need to store it.

The program can treat any missing position as zero.

---

47. Common beginner mistake 5: entering elements in random order

For now, avoid entering non-zero elements randomly.

Bad order example:

```
4 5 6
1 3 7
3 2 5
2 5 8
4 1 9
```

Better order:

```
1 3 7
2 5 8
3 2 5
```

```
4 1 9
4 5 6
```

The better order is row-major order.

This makes later display and addition algorithms easier.

---

48. Full lesson example

Matrix:

```
0 0 7 0 0
0 0 0 0 8
0 5 0 0 0
9 0 0 0 6
```

Matrix information:

```
m = 4
n = 5
num = 5
```

Coordinate List:

```
Row  Column  Value
1    3       7
2    5       8
3    2       5
4    1       9
4    5       6
```

Stored in memory:

```
e[0] = {1, 3, 7}
e[1] = {2, 5, 8}
e[2] = {3, 2, 5}
e[3] = {4, 1, 9}
e[4] = {4, 5, 6}
```

---

49. Practice example

Given this matrix:

```
0 4 0
0 0 0
7 0 9
```

Find:

```
m
n
num
Coordinate List
```

Solution:

The matrix has 3 rows and 3 columns.

So:

```
m = 3
n = 3
```

The non-zero values are:

```
4 at row 1, column 2
7 at row 3, column 1
9 at row 3, column 3
```

So:

```
num = 3
```

Coordinate List:

```
(1, 2, 4)
(3, 1, 7)
(3, 3, 9)
```

50. Another practice example

Given this matrix:

```
0 0 0 6
2 0 0 0
0 0 9 0
0 4 0 0
```

The matrix has:

```
m = 4
n = 4
```

The non-zero values are:

```
6 at row 1, column 4
2 at row 2, column 1
9 at row 3, column 3
4 at row 4, column 2
```

So:

```
num = 4
```

Coordinate List:

```
(1, 4, 6)
(2, 1, 2)
(3, 3, 9)
(4, 2, 4)
```

Stored array:

```
e[0] = {1, 4, 6}
e[1] = {2, 1, 2}
e[2] = {3, 3, 9}
e[3] = {4, 2, 4}
```

51. Important code symbols

Symbol	Meaning
`struct Element`	structure for one non-zero value
`struct Sparse`	structure for the whole sparse matrix
`m`	number of rows
`n`	number of columns
`num`	number of non-zero elements
`e`	pointer to the dynamic element array
`new Element[s->num]`	creates space for `num` elements
`&s`	address of `s`
`*s`	pointer variable used to access the original structure
`->`	used to access fields through a pointer
`.`	used to access fields through an object

52. Important formulas and ideas

Total positions in the full matrix:

```
m × n
```

Number of stored records:

```
num
```

Storage used by normal matrix:

```
O(m × n)
```

Storage used by Coordinate List:

```
O(num)
```

Creation time:

```
O(num)
```

Each non-zero value is stored as:

```
(row, column, value)
```

---

53. Lesson 2 final summary

A sparse matrix has mostly zero values.

A normal 2D array stores every value, including zeros.

Coordinate List representation stores only non-zero values.

Each non-zero value is stored as:

```
row, column, value
```

The `Element` structure stores one non-zero item:

```
struct Element {
    int i;
    int j;
    int x;
};
```

The `Sparse` structure stores the whole sparse matrix:

```
struct Sparse {
    int m;
    int n;
    int num;
```

```
    struct Element *e;
};
```

The field meanings are:

```
m = total rows
n = total columns
num = number of non-zero elements
e = dynamic array of Element records
```

The `create()` function:

```
reads m and n
reads num
allocates num Element records
reads each row, column, and value
stores the non-zero elements
```

The key memory idea is:

```
Do not store zeros.
Store only the useful values and their positions.
```

Complexity:

```
Creation time = O(num)
Storage space = O(num)
```

This lesson prepares you for the next step: displaying a sparse matrix as a full grid while still storing only the non-zero elements.

---

54. Exit questions

Question 1

What does `num` mean?

Answer:

```
num is the number of non-zero elements.
```

Question 2

Why do we store row and column numbers?

Answer:

```
Because non-zero values in a sparse matrix can appear anywhere.
The computer needs to know each value's position.
```

Question 3

Why do we allocate `new Element[s->num]` instead of `new Element[s->m * s->n]`?

Answer:

```
Because we only store non-zero elements, not the full matrix.
```

Question 4

What does this record mean?

```
(3, 2, 5)
```

Answer:

```
At row 3, column 2, the value is 5.
```

Question 5

Inside `create(struct Sparse *s)`, should we use `s.m` or `s->m`?

Answer:

```
Use s->m because s is a pointer.
```

---

55. One-sentence memory trick

A Coordinate List sparse matrix stores only this:

```
Where is the value, and what is the value?
```

That means:

```
row, column, value
```